Lab 1: Mastering Auto Loader – Initial Streaming Data Load

Lab Name

Mastering Auto Loader – Initial Streaming Data Load

Lab Summary

In this lab, we learn how to use **Databricks Auto Loader** to ingest files using **Spark Structured Streaming**. Auto Loader automatically detects new files arriving in cloud storage and processes them incrementally.

We will:

- Create a **workspace folder and notebook**
- Create a **volume for storing raw files**
- Upload a **CSV file**
- Configure **Auto Loader streaming query**
- Store schema and checkpoint metadata
- Write ingested data to **Delta format**

This lab demonstrates the **initial load using Auto Loader**.

Prerequisites

- Databricks Workspace access
- Unity Catalog enabled
- Databricks Bootcamp workspace folder created
- Bronze layer schema available (`databricks-bronze` or similar)
- Sample CSV file (example: `orders_batch1.csv`)

Steps to Complete Lab

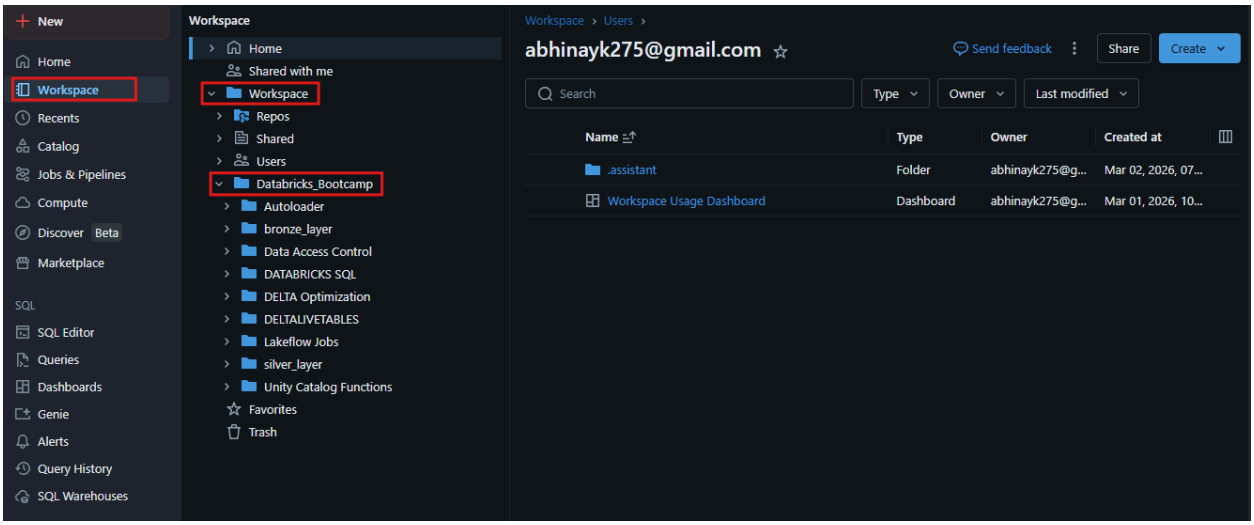
Task 1 – Create Workspace Folder for Auto Loader

Sub-task 1.1 – Navigate to Workspace

1. Open **Databricks Workspace**
2. Click **Workspace**
3. Navigate to your project folder

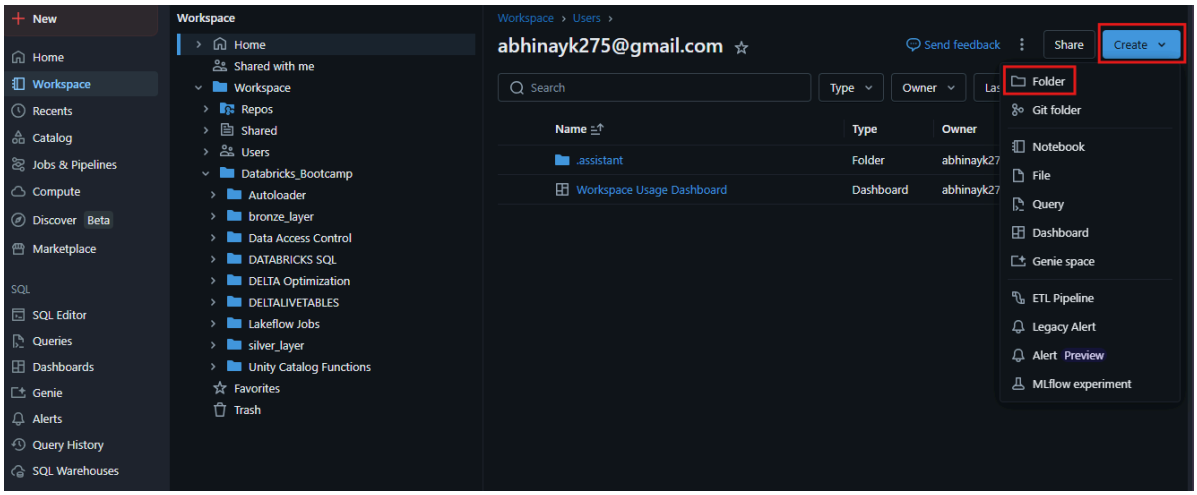
   Example:

```
Workspace → Databricks_Bootcamp
```



Sub-task 1.2 – Create Folder

1. Click **Create**
2. Select **Folder**



3. Name the folder:

```
Autoloader
```

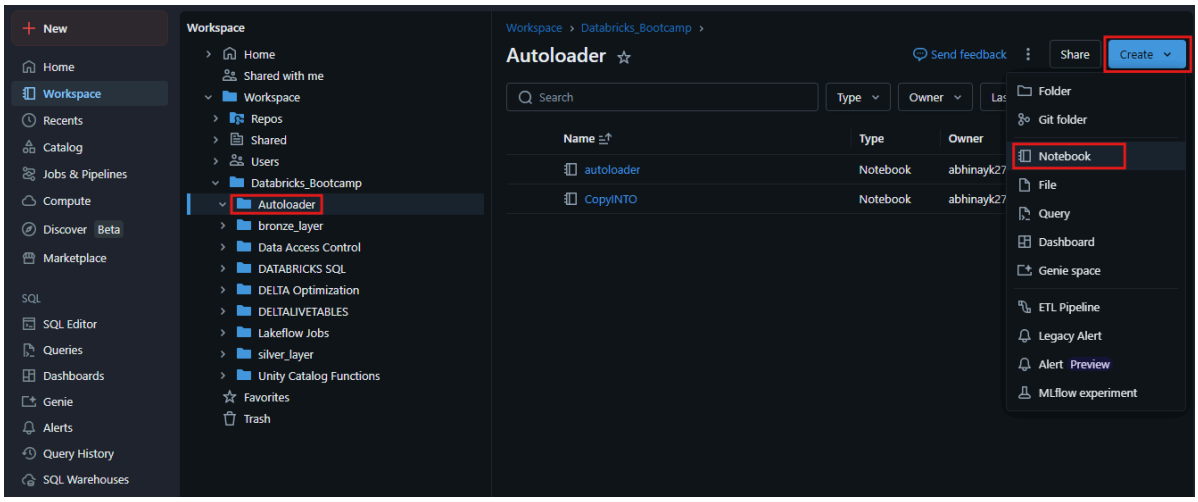
4. Click **Create**

This folder will store Auto Loader notebooks.

Task 2 – Create Auto Loader Notebook

Sub-task 2.1 – Create Notebook

1. Inside the **autoloader** folder
2. Click **Create**
3. Select **Notebook**



Provide the name:

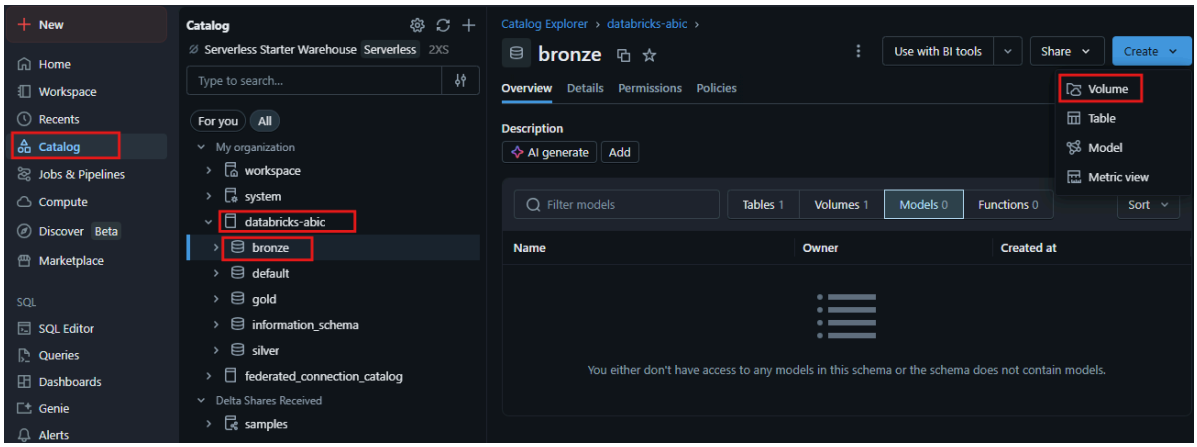
```
autoloader
```

Now a new notebook is created.

Task 3 – Create Volume for Raw Files

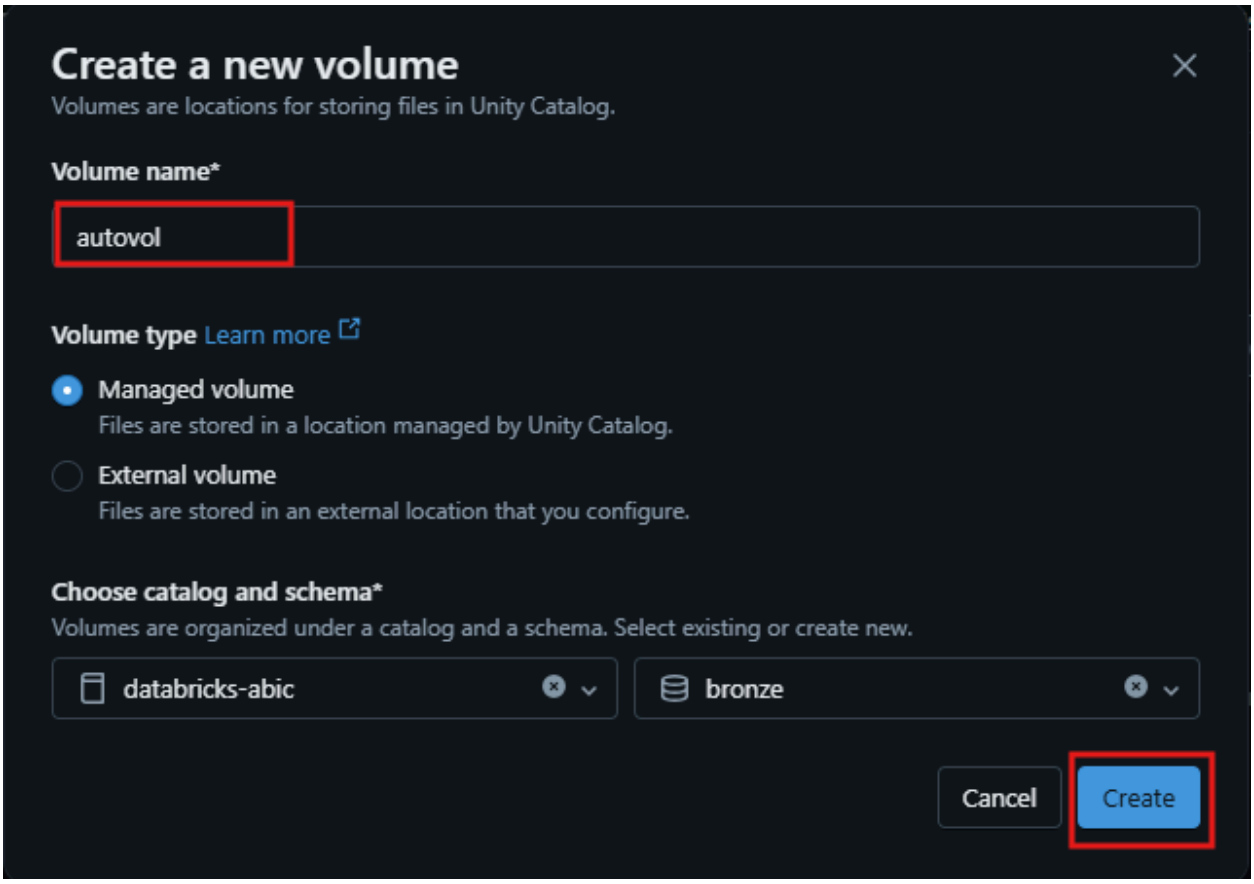
Sub-task 3.1 – Navigate to Catalog

1. Go to **Catalog**
2. Select the **Bronze schema**
3. Click **Create** → **Volume**



4. Volume Name:

```
autovol
```

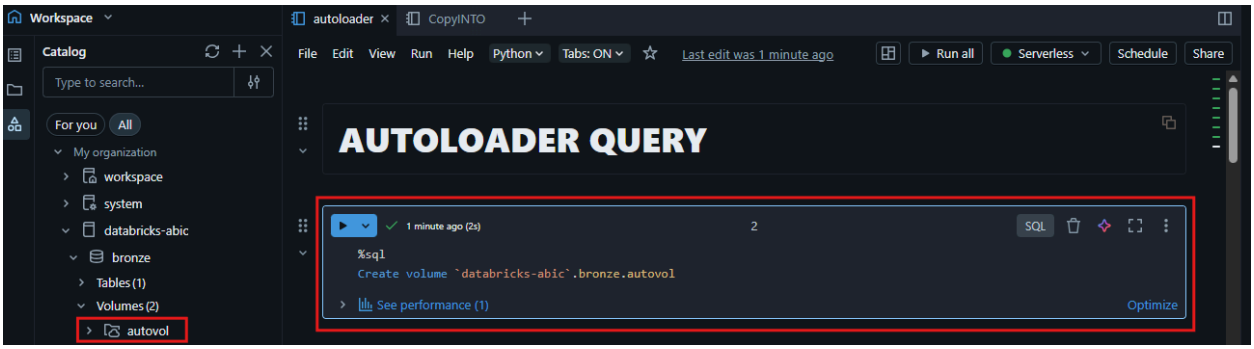


OR

Sub-task 3.1 – Create Volume

Create the volume by running code below in notebook:

```
Create volume `databricks-abic`.bronze.autovol
```



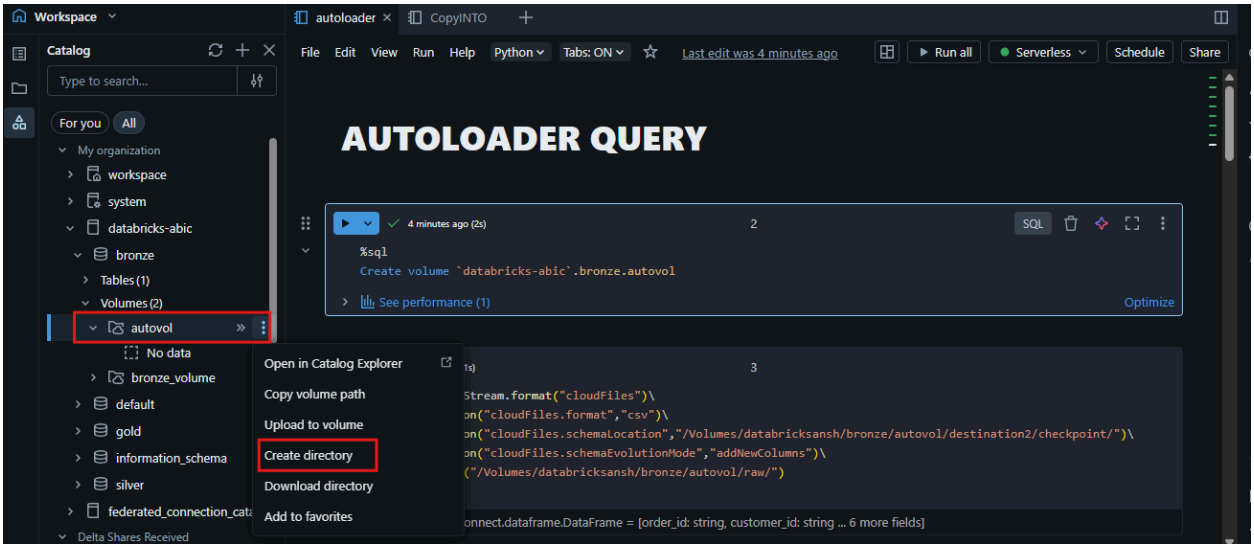
This volume will store **incoming raw files**.

Task 4 – Upload Source Data

Sub-task 4.1 – Create Raw Folder

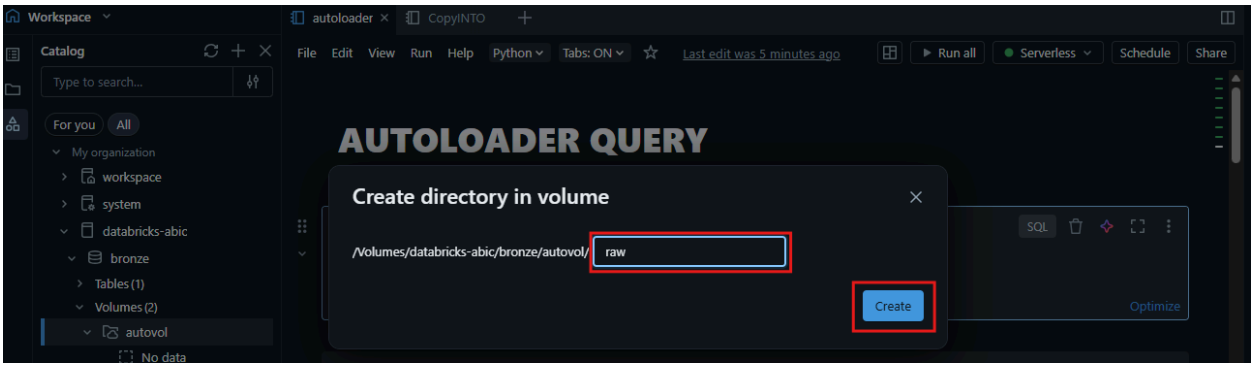
Inside the volume:

```
Volumes → bronze → autovol
```



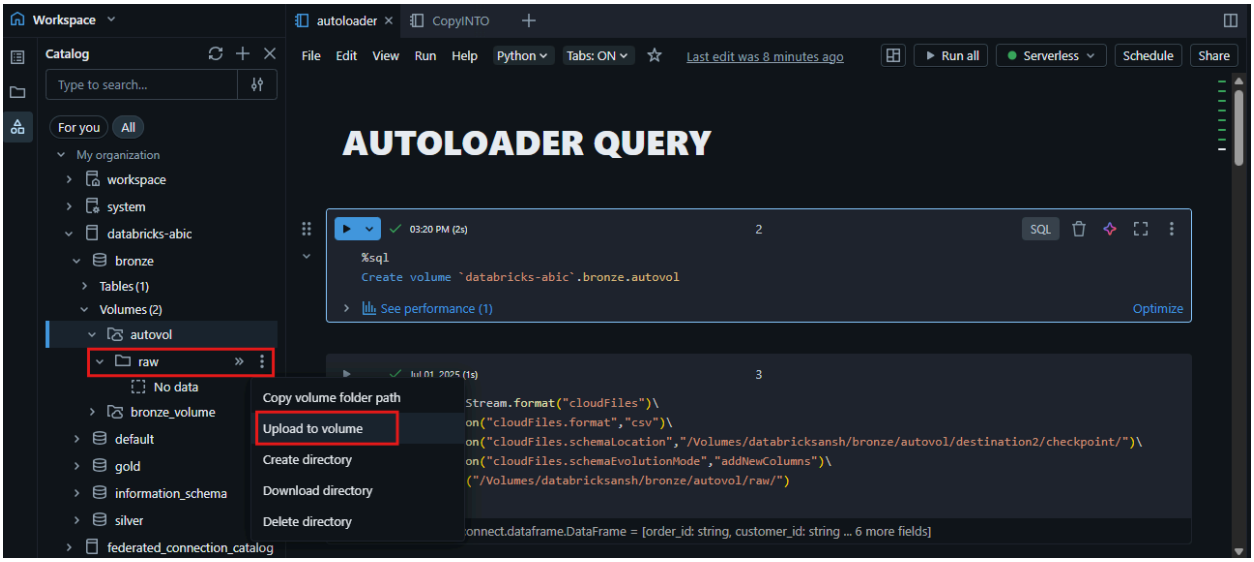
Create a new directory:

```
raw
```

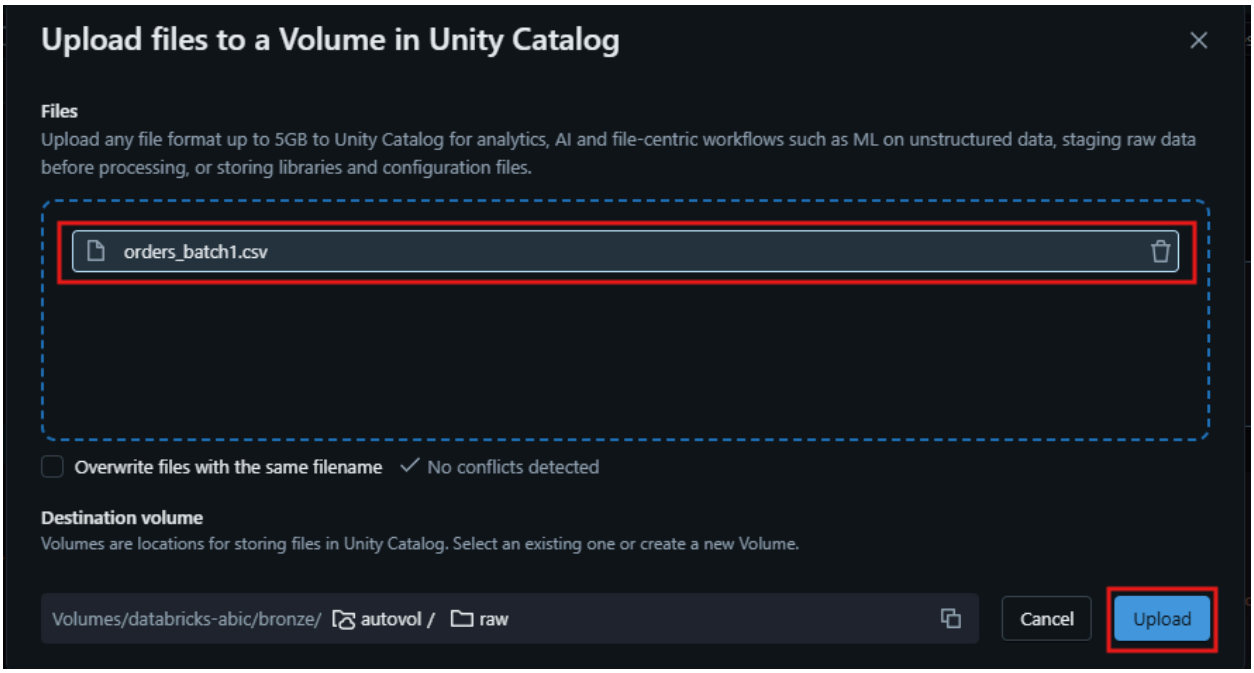


Sub-task 4.2 – Upload CSV File

Upload a file inside raw folder:




```
orders_batch1.csv
```

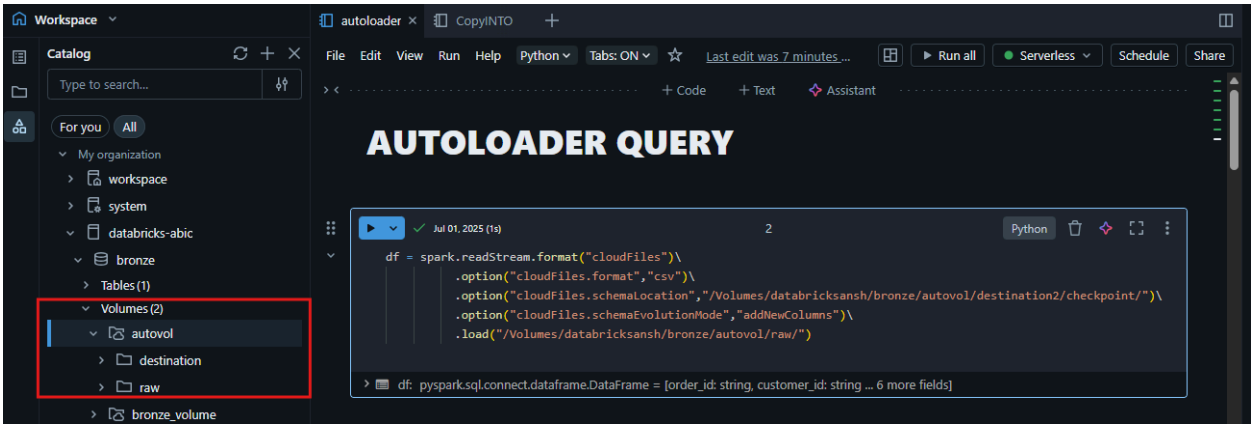


This file will act as the **initial dataset** for Auto Loader.

Task 5 – Create Destination Folders

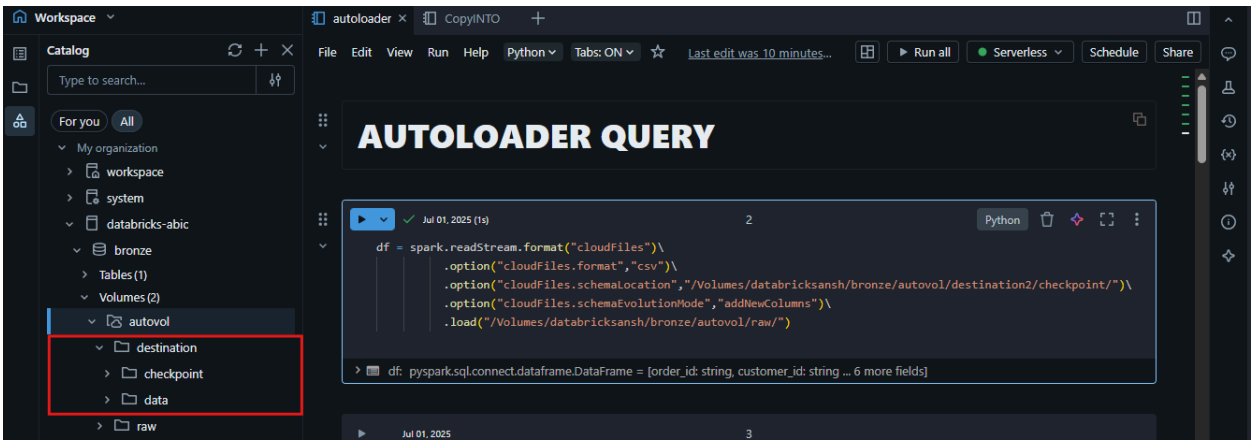
Inside the volume create a **destination folder**.

```
autovol
    ├── raw
    └── destination
```



Inside **destination**, create two folders:

```
destination
    ├── checkpoint
    └── data
```



Purpose of folders

Folder	Purpose
checkpoint	Stores streaming state
data	Stores processed Delta data

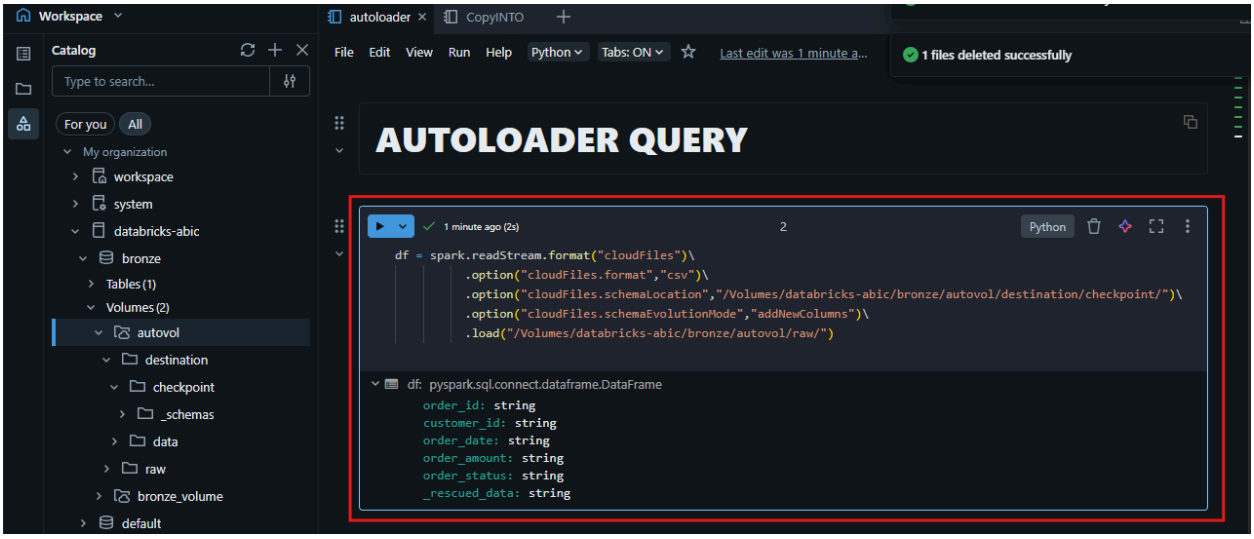
Task 6 – Create Auto Loader Streaming Query

Add the following code to the notebook.

Auto Loader Query

```
df = spark.readStream.format("cloudFiles")\
          .option("cloudFiles.format","csv")\
          .option("cloudFiles.schemaLocation","/Volumes/datab
ricks-abic/bronze/autovol/destination/checkpoint/")\
          .option("cloudFiles.schemaEvolutionMode","addNewCol
```

```
umns")\
          .load("/Volumes/databricks-abic/bronze/autovol/raw/")
```



Explanation

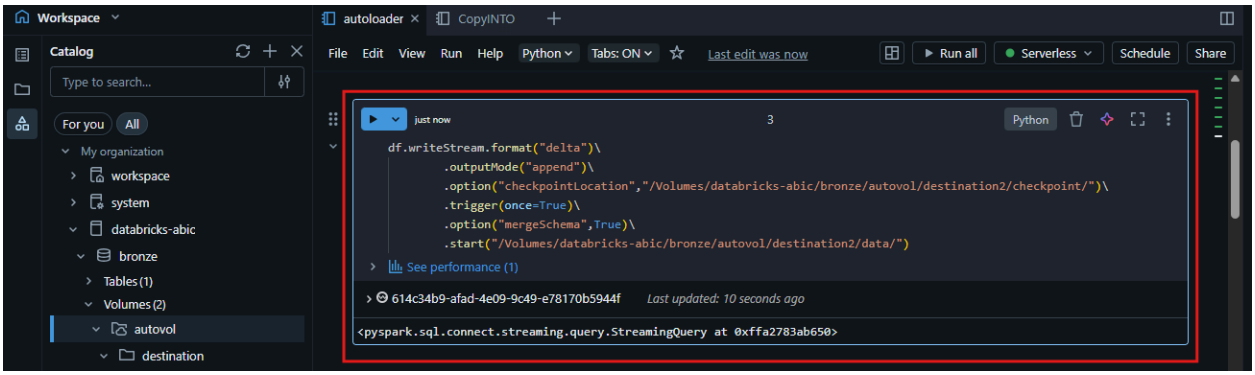
Component	Description
readStream	Enables streaming ingestion
cloudFiles	Auto Loader format
cloudFiles.format	Specifies file type
schemaLocation	Stores schema metadata
schemaEvolutionMode	Handles schema changes
load()	Source file location

Task 7 – Configure Streaming Write

Now write the data to Delta format.

```
df.writeStream.format("delta")\
        .outputMode("append")\
```

```
        .option("checkpointLocation","/Volumes/databricks-abi
c/bronze/autovol/destination/checkpoint/")\
        .trigger(once=True)\
        .option("mergeSchema",True)\
        .start("/Volumes/databricks-abic/bronze/autovol/desti
nation/data/")
```



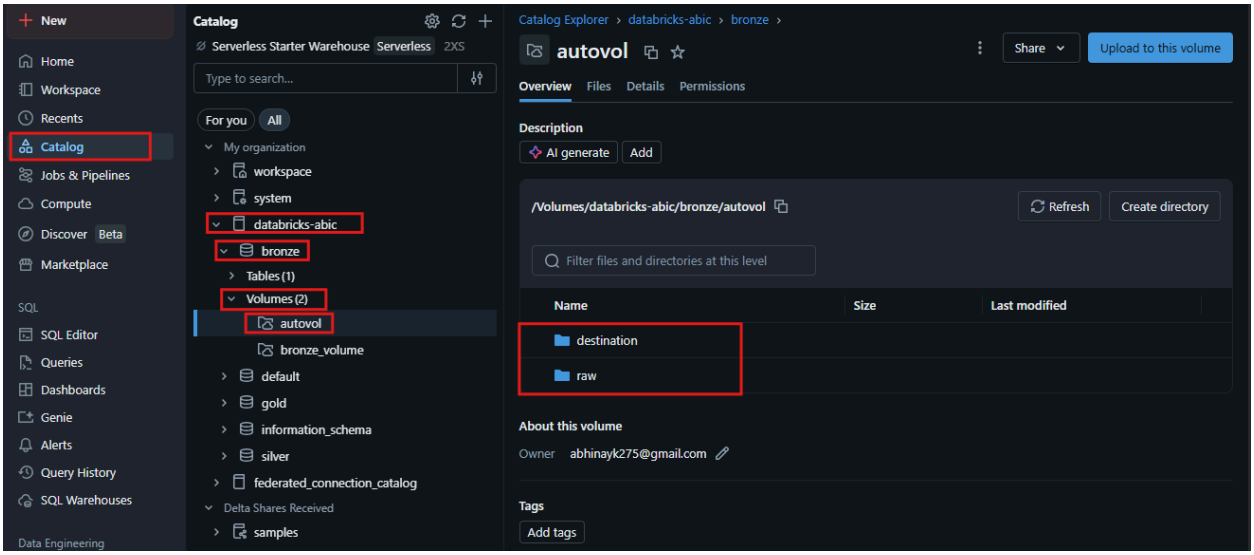
Explanation

Option	Description
writeStream	Streaming write
format("delta")	Writes data as Delta table
outputMode("append")	Appends new data
checkpointLocation	Stores streaming progress
trigger(availableNow=True)	Runs once and stops
start()	Destination path

Task 8 – Verify Output

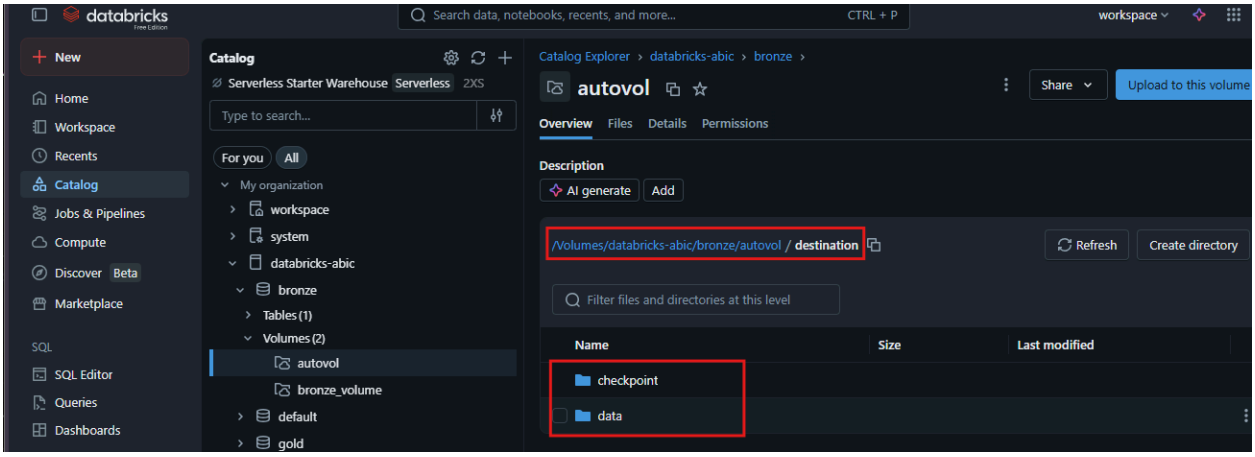
Navigate to Catalog:

```
bronze → Volumes → autovol → destination
```



You will see:

```
destination
    ├── checkpoint
    └── data
```

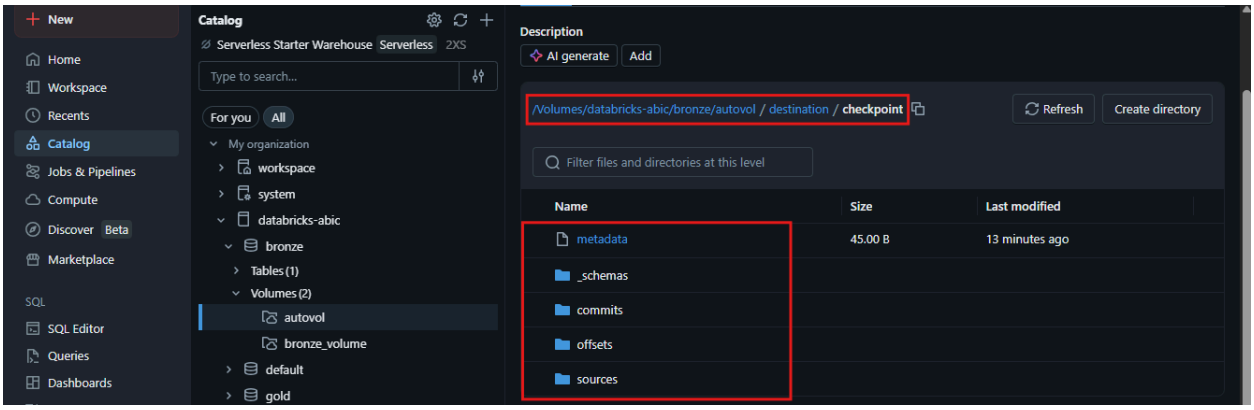


Checkpoint Folder Structure

Inside checkpoint you will see:

```
checkpoint
    ├── commits
    ├── offsets
```

```
    ├── sources
    └── schemas
```



schemas

Stores **Auto Loader schema metadata**.

sources

Maintains **idempotency information** using RocksDB.

Tracks which files are:

- processed
- not processed

offsets

Stores **streaming offsets** used for tracking progress.

commits

Confirms successfully processed files.

Acts like a **commit marker**.

Outcome

After completing this lab, you will be able to:

- Create Auto Loader notebooks

- Upload source files
- Configure streaming ingestion
- Use schema evolution modes
- Manage checkpoint and schema storage
- Write streaming data to Delta format